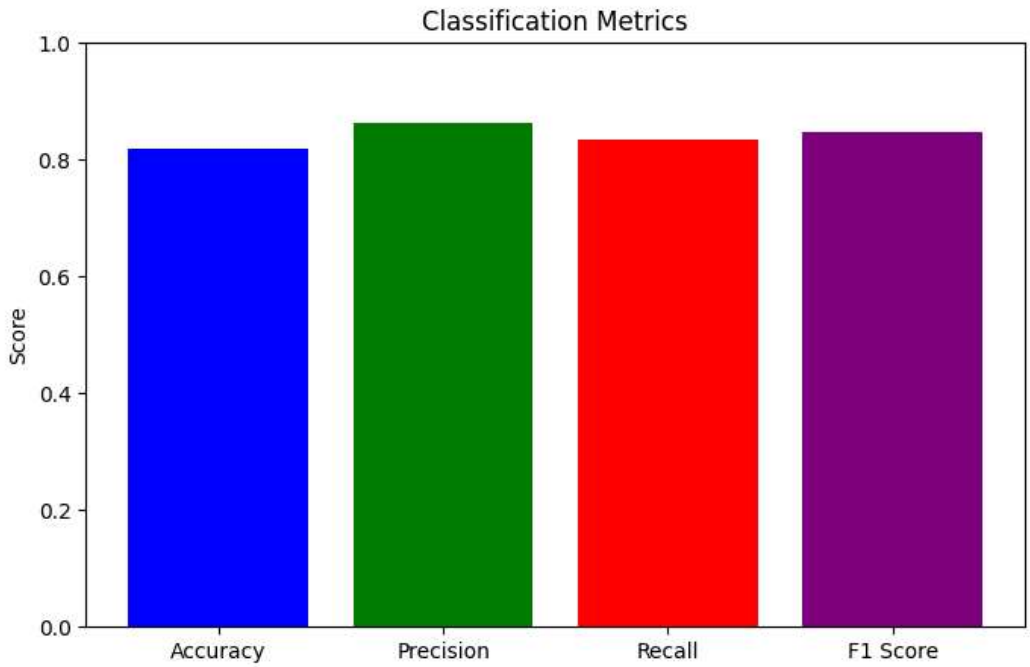
Problem 1: Confusion Matrix Metrics (Python as a Calculator) A binary classification model predicts whether rainfall will cause Flood (Yes/No). Given: • True Positives (TP) = 50 • False Positives (FP) = 8 • True Negatives (TN) = 32 • False Negatives (FN) = 10

```
import matplotlib.pyplot as plt

TP = 50  # True Positives
FP = 8   # False Positives
TN = 32  # True Negatives
FN = 10  # False Negatives

# Calculate total predictions
total_predictions = TP + FP + TN + FN

# Calculate accuracy
accuracy = (TP + TN) / (TP + FP + TN + FN)

# Calculate precision
precision = TP / (TP + FP)

# Calculate recall
recall = TP / (TP + FN)

# Calculate F1 score
f1_score = (2 * precision * recall) / (precision + recall)

# Print results with clear labels
print("Total Predictions:", total_predictions)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1_score)

# Prepare data for bar chart
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
values = [accuracy, precision, recall, f1_score]

# Create bar chart
plt.figure(figsize=(8, 5))
plt.bar(metrics, values, color=['blue', 'green', 'red', 'purple'])
plt.ylim(0, 1) # Metrics are usually between 0 and 1
plt.title('Classification Metrics')
plt.ylabel('Score')
plt.show()
```

```
Total Predictions: 100
Accuracy: 0.82
Precision: 0.8620689655172413
Recall: 0.8333333333333334
F1 Score: 0.847457627118644
```



Problem 2: R^2 (Coefficient of Determination) In a regression model predicting river discharge, the following are known: • Sum of Squares due to Regression (SSR) = 420 • Sum of Squares due to Error (SSE) = 180

```
# Assign given values
SSR = 420  # Sum of Squares due to Regression
SSE = 180  # Sum of Squares due to Error

# Calculate Total Sum of Squares (SST)
SST = SSR + SSE

# Calculate R-squared
R_squared = SSR / SST

# Print calculated values with clear labels
print("Total Sum of Squares (SST):", SST)
print("R-squared (R^2):", R_squared)

# Interpret whether the model explains more than 70% of the variance
if R_squared > 0.7:
    print("Interpretation: The model explains more than 70% of the variance.")
else:
    print("Interpretation: The model does NOT explain more than 70% of the variance.")
```

```
Total Sum of Squares (SST): 600
R-squared (R^2): 0.7
Interpretation: The model does NOT explain more than 70% of the variance.
```

Problem 3: Mean Squared Error (MSE) Using Python as Calculator A model predicts temperature values. The squared errors for 5 observations are: (2,4,1,9,4)

```
# Store squared errors in individual variables
e1 = 2
e2 = 4
e3 = 1
```

```
e4 = 9
e5 = 4

# Number of observations
n = 5

# Calculate the sum of squared errors
sum_squared_errors = e1 + e2 + e3 + e4 + e5

# Calculate Mean Squared Error (MSE)
MSE = sum_squared_errors / n

# Print the result with a clear label
print("Mean Squared Error (MSE):", MSE)
```

```
Mean Squared Error (MSE): 4.0
```

Problem 4: Cook's Distance (Influential Observation) For a regression model: • Squared residual (e_i^2) = 9 • Mean Squared Error (MSE) = 3 • Number of predictors (p) = 3 • Leverage value (h_{ii}) = 0.2 • Sample size (n) = 25

```
# Assign given values
e_squared = 9     # Squared residual (e_i^2)
MSE = 3           # Mean Squared Error
p = 3             # Number of predictors
h_ii = 0.2        # Leverage value
n = 25            # Sample size

# Calculate Cook's Distance
# D_i = (e_i^2 / (p * MSE)) * (h_ii / (1 - h_ii)^2)
cooks_distance = (e_squared / (p * MSE)) * (h_ii / ((1 - h_ii) ** 2))

# Calculate threshold value (4 / n)
threshold = 4 / n

# Print results with clear labels
print("Cook's Distance:", cooks_distance)
print("Threshold (4/n):", threshold)

# Decide whether the observation is influential
if cooks_distance > threshold:
    print("Decision: The observation IS influential.")
else:
    print("Decision: The observation is NOT influential.")
```

```
Cook's Distance: 0.31249999999999994
Threshold (4/n): 0.16
Decision: The observation IS influential.
```

Problem 5: Simple Linear Regression Prediction (Calculator Style) A regression model for predicting crop yield is: y=2.5x+10

```
# Assign given values
slope = 2.5        # Slope of the regression line
intercept = 10     # Intercept of the regression line
x = 12             # Given rainfall value

# Calculate predicted value of y using y = mx + b
y = slope * x + intercept

# Print the predicted output
print("Predicted value of y:", y)

# Explain the meaning of slope and intercept
```

```
print("Explanation:")
print("The slope (2.5) means that for each 1-unit increase in rainfall,")
print("the crop yield increases by 2.5 units.")
print("The intercept (10) represents the crop yield when rainfall is zero.")
```

```
Predicted value of y: 40.0
Explanation:
The slope (2.5) means that for each 1-unit increase in rainfall,
the crop yield increases by 2.5 units.
The intercept (10) represents the crop yield when rainfall is zero.
```